

# CSE 713: Artificial Intelligence

## Search Algorithms — Complete Model Answers

University of Chittagong — 7<sup>th</sup> Semester B.Sc. (Engg.) Examination

Covers Years: 2024, 2023, 2022, 2021, 2020 — Topics: UCS, BFS, DFS, Greedy, A\*,  
IDDFS, IDA\*

## Contents

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>1 Examination 2024 — Q2 (9 Marks)</b>                                     | <b>2</b>  |
| 1.1 Problem Data . . . . .                                                   | 2         |
| 1.2 Part a(i) — State-Space Graph [1 mark] . . . . .                         | 2         |
| 1.3 Part a(ii) — Search Algorithm Traces [5 marks] . . . . .                 | 3         |
| 1.4 Part b — Problem Formulations [3 marks] . . . . .                        | 8         |
| <b>2 Examination 2023 — Q2 (9 Marks)</b>                                     | <b>10</b> |
| 2.1 Problem Data . . . . .                                                   | 10        |
| 2.2 Part a — Problem Formulations [2 marks] . . . . .                        | 10        |
| 2.3 Part b(i) — State-Space Diagram [0.5 mark] . . . . .                     | 11        |
| 2.4 Part b(ii) — Search Algorithm Traces [4.5 marks] . . . . .               | 12        |
| 2.5 Part c — IDDFS: Synthesising Benefits of BFS and DFS [2 marks] . . . . . | 15        |
| <b>3 Examination 2022</b>                                                    | <b>18</b> |
| 3.1 Section A — Q.A-4(a): DFS vs BFS [2 marks] . . . . .                     | 18        |
| 3.2 Section B — Q.B-4(b): Graph Trace [3 marks] . . . . .                    | 21        |

## Examination 2024 — Q2 (9 Marks)

### Problem Data

**States:** A (start), B, C, D, E, F, H, J, K, G (goal). All roads bidirectional.

**Table 1: Road Connections and Step Costs**

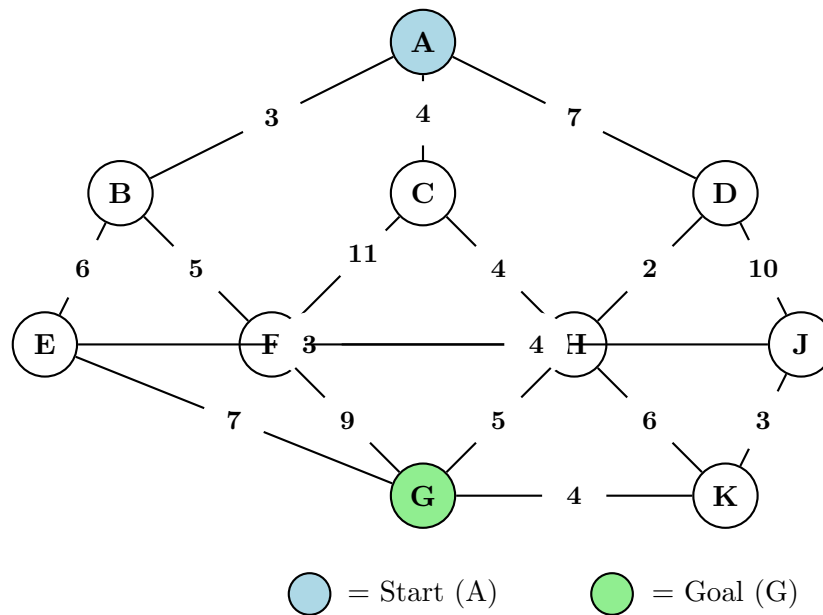
| From | To | Cost |
|------|----|------|
| A    | B  | 3    |
| A    | C  | 4    |
| A    | D  | 7    |
| B    | E  | 6    |
| B    | F  | 5    |
| C    | F  | 11   |
| C    | H  | 4    |
| D    | H  | 2    |
| D    | J  | 10   |
| E    | G  | 7    |
| E    | H  | 3    |
| F    | G  | 9    |
| F    | J  | 4    |
| H    | G  | 5    |
| H    | K  | 6    |
| J    | K  | 3    |
| K    | G  | 4    |

**Table 2: Heuristic Estimates  $h(n)$  to Goal G**

| Node | $h(n)$ |
|------|--------|
| A    | 11     |
| B    | 10     |
| C    | 7      |
| D    | 6      |
| E    | 6      |
| F    | 6      |
| H    | 4      |
| J    | 5      |
| K    | 3      |
| G    | 0      |

### Part a(i) — State-Space Graph [1 mark]

**Question:** Using Table 1, draw the state-space graph clearly labelling all nodes and edge costs.



**Note:** The edge E–H is curved upward and F–J is curved downward to avoid visual ambiguity where nodes share the same  $y$ -coordinate. All edges are **bidirectional** (undirected).

### Part a(ii) — Search Algorithm Traces [5 marks]

**Question:** Show search trees for (1) UCS, (2) Greedy Best-First, (3) A\*, (4) IDDFS from A to G. At every step: show node being expanded and fringe contents in order of selection. For IDDFS: show depth limit per iteration. Ties broken alphabetically.

### Algorithm 1 — Uniform-Cost Search (UCS)

**Key idea:** Always expand the node with the **lowest path cost**  $g(n)$  from the start. The fringe is a **min-priority queue sorted by**  $g(n)$ . A node is closed (never re-expanded) once popped. If a *cheaper* path to a fringe node is found, its cost is updated (replaced).

**Fringe notation:** Node[ $g$ ] sorted ascending by  $g$ .

| Step | Node Expanded | $g(n)$ | Fringe after expansion (sorted by $g$ )                                                                                                                                                         |
|------|---------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>A</b>      | 0      | B[3], C[4], D[7]<br>Neighbours of A: B( $g=3$ ), C( $g=4$ ), D( $g=7$ ).                                                                                                                        |
| 2    | <b>B</b>      | 3      | C[4], D[7], F[8], E[9]<br>A already closed. $B \rightarrow F$ : $g=8$ ; $B \rightarrow E$ : $g=9$ . Both new.                                                                                   |
| 3    | <b>C</b>      | 4      | D[7], F[8], H[8], E[9]<br>$C \rightarrow F$ : $g=15 > 8$ already in fringe $\Rightarrow$ keep F[8]. $C \rightarrow H$ : $g=8$ new.                                                              |
| 4    | <b>D</b>      | 7      | F[8], H[8], E[9], J[17]<br>$D \rightarrow H$ : $g=9 > 8$ in fringe $\Rightarrow$ keep H[8]. $D \rightarrow J$ : $g=17$ new.<br>Tie F,H: <b>F before H</b> (alphabetical).                       |
| 5    | <b>F</b>      | 8      | H[8], E[9], J[12], G[17]<br>$F \rightarrow G$ : $g=17$ new. $F \rightarrow J$ : $g=12 < 17$ in fringe $\Rightarrow$ <b>update</b> J[12].                                                        |
| 6    | <b>H</b>      | 8      | E[9], J[12], G[13], K[14]<br>$H \rightarrow E$ : $g=11 > 9$ in fringe $\Rightarrow$ keep E[9]. $H \rightarrow G$ : $g=13 < 17 \Rightarrow$ <b>update</b> G[13]. $H \rightarrow K$ : $g=14$ new. |
| 7    | <b>E</b>      | 9      | J[12], G[13], K[14]<br>$E \rightarrow G$ : $g=16 > 13$ in fringe $\Rightarrow$ keep G[13].                                                                                                      |
| 8    | <b>J</b>      | 12     | G[13], K[14]<br>$J \rightarrow K$ : $g=15 > 14$ in fringe $\Rightarrow$ keep K[14].                                                                                                             |
| 9    | <b>G</b>      | 13     | <b>GOAL FOUND</b>                                                                                                                                                                               |

**UCS Solution:**  $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$  **Total cost:**  $4 + 4 + 5 = 13$   
 Nodes expanded (in order): A, B, C, D, F, H, E, J, G (9 nodes total)  
 UCS **guarantees the optimal path** because it always expands the cheapest unexplored node, never re-expands a closed node, and path costs are non-negative.

### Algorithm 2 — Greedy Best-First Search

**Key idea:** Always expand the node with the **lowest heuristic estimate**  $h(n)$  to the goal. The fringe is sorted by  $h(n)$  (ascending). Path cost  $g(n)$  is *completely ignored*. Greedy is fast but not guaranteed to find the optimal path.

**Fringe notation:** Node[ $h$ ] sorted ascending by  $h$ .

| Step | Node Expanded | $h(n)$ | Fringe after expansion (sorted by $h$ )                                                                                                |
|------|---------------|--------|----------------------------------------------------------------------------------------------------------------------------------------|
| 1    | A             | 11     | D[6], C[7], B[10]<br>Neighbours B( $h=10$ ), C( $h=7$ ), D( $h=6$ ). Sorted by $h$ : D, C, B.                                          |
| 2    | D             | 6      | H[4], J[5], C[7], B[10]<br>D's neighbours excl. A: H( $h=4$ ), J( $h=5$ ).                                                             |
| 3    | H             | 4      | G[0], K[3], J[5], E[6], C[7], B[10]<br>H's neighbours excl. D: C( $h=7$ , already in fringe—skip), E( $h=6$ ), G( $h=0$ ), K( $h=3$ ). |
| 4    | G             | 0      | <b>GOAL FOUND</b>                                                                                                                      |

**Greedy Solution:**  $A \xrightarrow{7} D \xrightarrow{2} H \xrightarrow{5} G$       **Total cost:**  $7 + 2 + 5 = 14$  (not optimal!)

Nodes expanded: A, D, H, G (only 4 nodes — very fast)

Greedy expanded just 4 nodes versus UCS's 9, but found a **suboptimal path** (cost 14 vs optimal 13). It followed the heuristic blindly: D had the lowest  $h$  from A, leading it down the D–H–G route instead of the cheaper A–C–H–G route.

### Algorithm 3 — A\* Search

**Key idea:** Expand the node with the **lowest evaluation function**  $f(n) = g(n) + h(n)$ , where  $g(n)$  = exact cost from start and  $h(n)$  = heuristic estimate to goal. The fringe is sorted by  $f(n)$  (ascending). Ties in  $f$  broken alphabetically. If a cheaper path to a fringe node is found, update its entry.

**Fringe notation:** Node[ $g, h, f$ ] sorted ascending by  $f$ .

| Step | Node     | g  | h  | f  | Fringe after expansion (sorted by f)                                                                                                                                                                                                                                                                                |
|------|----------|----|----|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <b>A</b> | 0  | 11 | 11 | C[4,7, <b>11</b> ], B[3,10, <b>13</b> ], D[7,6, <b>13</b> ]<br>Tie B,D at $f=13$ : B first.                                                                                                                                                                                                                         |
| 2    | <b>C</b> | 4  | 7  | 11 | H[8,4, <b>12</b> ], B[3,10, <b>13</b> ], D[7,6, <b>13</b> ], F[15,6, <b>21</b> ]<br>$C \rightarrow H$ : $g=8, f=12$ new. $C \rightarrow F$ : $g=15, f=21$ new.                                                                                                                                                      |
| 3    | <b>H</b> | 8  | 4  | 12 | B[3,10, <b>13</b> ], D[7,6, <b>13</b> ], G[13,0, <b>13</b> ],<br>E[11,6, <b>17</b> ], K[14,3, <b>17</b> ], F[15,6, <b>21</b> ]<br>$H \rightarrow D$ : $f=16 > 13$ , keep. $H \rightarrow G$ : $f=13$ new.<br>$H \rightarrow E$ : $f=17$ new. $H \rightarrow K$ : $f=17$ new. Tie B,D,G at $f=13$ : <b>B first</b> . |
| 4    | <b>B</b> | 3  | 10 | 13 | D[7,6, <b>13</b> ], G[13,0, <b>13</b> ], F[8,6, <b>14</b> ], E[9,6, <b>15</b> ],<br>K[14,3, <b>17</b> ]<br>$B \rightarrow F$ : $g=8, f=14 < 21 \Rightarrow$ <b>update</b> F. $B \rightarrow E$ :<br>$g=9, f=15 < 17 \Rightarrow$ <b>update</b> E. Tie D,G at $f=13$ : <b>D first</b> .                              |
| 5    | <b>D</b> | 7  | 6  | 13 | G[13,0, <b>13</b> ], F[8,6, <b>14</b> ], E[9,6, <b>15</b> ],<br>K[14,3, <b>17</b> ], J[17,5, <b>22</b> ]<br>$D \rightarrow J$ : $g=17, f=22$ new.                                                                                                                                                                   |
| 6    | <b>G</b> | 13 | 0  | 13 | <b>GOAL FOUND</b>                                                                                                                                                                                                                                                                                                   |

**A\* Solution:**  $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$       **Total cost:**  $4 + 4 + 5 = 13$  (optimal!)  
 Nodes expanded: A, C, H, B, D, G (6 nodes — fewer than UCS's 9)  
 A\* is both **complete** and **optimal** when the heuristic is *admissible* ( $h(n) \leq h^*(n)$ , i.e. it never overestimates). It is more efficient than UCS because  $h(n)$  guides the search toward promising nodes, pruning unpromising directions early.

#### Algorithm 4 — Iterative Deepening DFS (IDDFS)

**Key idea:** Perform DFS with a depth cutoff  $L = 0, 1, 2, \dots$ , restarting from the root each time, until the goal is found. The “fringe” is a **stack** (LIFO). Children are pushed in *reverse* alphabetical order so they are *popped* (and expanded) in alphabetical order. Nodes on the **current path** (ancestors) are not revisited to avoid infinite loops.

**Stack notation:** Leftmost entry = top of stack (next to be popped). Depth shown in parentheses.

**Iteration**  $L = 0$ :

| Step                                      | Action                                                           | Stack (top $\leftarrow$ left) |
|-------------------------------------------|------------------------------------------------------------------|-------------------------------|
| 1                                         | Pop <b>A</b> (d=0). $A \neq G$ . At depth limit — cannot expand. | [ ]                           |
| <i>No solution at <math>L = 0</math>.</i> |                                                                  |                               |

Iteration  $L = 1$ :

| Step                                      | Action                                             | Stack            |
|-------------------------------------------|----------------------------------------------------|------------------|
| 1                                         | Pop <b>A</b> (d=0). Expand: push D,C,B (B on top). | B(1), C(1), D(1) |
| 2                                         | Pop <b>B</b> (d=1). $B \neq G$ . At depth limit.   | C(1), D(1)       |
| 3                                         | Pop <b>C</b> (d=1). $C \neq G$ . At depth limit.   | D(1)             |
| 4                                         | Pop <b>D</b> (d=1). $D \neq G$ . At depth limit.   | [ ]              |
| <i>No solution at <math>L = 1</math>.</i> |                                                    |                  |

Iteration  $L = 2$ :

| Step                                      | Action                                               | Stack                  |
|-------------------------------------------|------------------------------------------------------|------------------------|
| 1                                         | Pop <b>A</b> (d=0). Expand: B,C,D.                   | B(1), C(1), D(1)       |
| 2                                         | Pop <b>B</b> (d=1). Expand (excl. A): E,F. Push F,E. | E(2), F(2), C(1), D(1) |
| 3                                         | Pop <b>E</b> (d=2). $E \neq G$ . At limit.           | F(2), C(1), D(1)       |
| 4                                         | Pop <b>F</b> (d=2). At limit.                        | C(1), D(1)             |
| 5                                         | Pop <b>C</b> (d=1). Expand (excl. A): F,H. Push H,F. | F(2), H(2), D(1)       |
| 6                                         | Pop <b>F</b> (d=2). At limit.                        | H(2), D(1)             |
| 7                                         | Pop <b>H</b> (d=2). At limit.                        | D(1)                   |
| 8                                         | Pop <b>D</b> (d=1). Expand (excl. A): H,J. Push J,H. | H(2), J(2)             |
| 9                                         | Pop <b>H</b> (d=2). At limit.                        | J(2)                   |
| 10                                        | Pop <b>J</b> (d=2). At limit.                        | [ ]                    |
| <i>No solution at <math>L = 2</math>.</i> |                                                      |                        |

Iteration  $L = 3$ :

| Step | Action                                                                     | Stack                  |
|------|----------------------------------------------------------------------------|------------------------|
| 1    | Pop <b>A</b> (d=0). Expand: B,C,D.                                         | B(1), C(1), D(1)       |
| 2    | Pop <b>B</b> (d=1). Expand (excl. A): E,F.                                 | E(2), F(2), C(1), D(1) |
| 3    | Pop <b>E</b> (d=2). Expand (excl. B): G,H. Push H first then G (G on top). | G(3), H(3), F(2), C(1) |
| 4    | Pop <b>G</b> (d=3). $G = \text{GOAL}$ .                                    | <b>GOAL FOUND</b>      |

**IDDFS Solution:**  $A \xrightarrow{3} B \xrightarrow{6} E \xrightarrow{7} G$       **Depth: 3**      **Cost:**  $3 + 6 + 7 = 16$

IDDFS found the solution at depth-limit  $L = 3$  in just 4 node expansions during the final iteration.

**Important distinction:** IDDFS finds the *shallowest* path (fewest hops), **not** the lowest-cost path. The optimal path  $A \rightarrow C \rightarrow H \rightarrow G$  also has depth 3, but alphabetical DFS expansion reaches  $G$  via  $B \rightarrow E \rightarrow G$  first. For non-uniform step costs, IDDFS is **not optimal**. It is however **complete** — it will always find a solution if one exists.

**Why IDDFS over BFS?** IDDFS uses only  $O(bL)$  space (a single stack), whereas BFS requires  $O(b^L)$  space to store all frontier nodes. The repeated work is a constant factor overhead.

### Algorithm Comparison Summary:

| Algorithm | Path Found                                    | Cost | Nodes Expanded     | Optimal?               |
|-----------|-----------------------------------------------|------|--------------------|------------------------|
| UCS       | $A \rightarrow C \rightarrow H \rightarrow G$ | 13   | 9 (all iterations) | <b>Yes</b>             |
| Greedy    | $A \rightarrow D \rightarrow H \rightarrow G$ | 14   | 4                  | No                     |
| $A^*$     | $A \rightarrow C \rightarrow H \rightarrow G$ | 13   | 6                  | <b>Yes</b>             |
| IDDFS     | $A \rightarrow B \rightarrow E \rightarrow G$ | 16   | 4 (final iter.)    | No (non-uniform costs) |

### Part b — Problem Formulations [3 marks]

**Question:** Give the initial state, goal test, operators, and path cost function for each problem.

#### (i) Robot Vacuum Cleaning Problem (Indoor Environment)

**State:** A pair  $(l, D)$  where  $l$  is the robot's current room and  $D \subseteq \{\text{all rooms}\}$  is the set of dirty rooms. *Example:* (Kitchen, {Bedroom, Lounge}).

**Initial state:** (start room, {all currently dirty rooms}).

**Goal test:**  $D = \emptyset$  (no dirty rooms remain).

**Operators:**

1.  $\text{Move}(r_i \rightarrow r_j)$ : Move from room  $r_i$  to adjacent room  $r_j$  (changes location;  $D$  unchanged).
2.  $\text{Suck}()$ : Vacuum the current room (removes current room from  $D$  if dirty).

Static obstacles (furniture) constrain which rooms are adjacent.

**Path cost:** Total energy used: each Move costs  $c_{\text{move}}$  energy units; each Suck costs  $c_{\text{suck}}$  units. Goal: minimise total energy.

**Informed vs. Uninformed:** **Informed search ( $A^*$ ) is more suitable.** A strong heuristic  $h(n) = |D| \times \bar{d}$  (number of remaining dirty rooms multiplied



by average travel cost between rooms) estimates remaining work without overestimating. With many rooms, uninformed BFS/UCS must explore exponentially many  $(l, D)$  state combinations blindly. A\* with a good heuristic dramatically prunes this search space.

## (ii) Water Jug Problem (5L and 3L Jugs)

**State:** A pair  $(x, y)$  where  $x \in \{0, 1, 2, 3, 4, 5\}$  is water (litres) in the 5L jug and  $y \in \{0, 1, 2, 3\}$  is water in the 3L jug.

**Initial state:**  $(0, 0)$  — both jugs empty.

**Goal test:**  $x = 4$  (exactly 4 litres in the 5L jug). The 3L jug cannot hold 4L, so only  $x = 4$  satisfies the goal.

**Operators:**

1. **Fill 5L:**  $(x, y) \rightarrow (5, y)$
2. **Fill 3L:**  $(x, y) \rightarrow (x, 3)$
3. **Empty 5L:**  $(x, y) \rightarrow (0, y)$
4. **Empty 3L:**  $(x, y) \rightarrow (x, 0)$
5. **Pour 5L  $\rightarrow$  3L:**  $(x, y) \rightarrow (x-d, y+d)$  where  $d = \min(x, 3-y)$
6. **Pour 3L  $\rightarrow$  5L:**  $(x, y) \rightarrow (x+d, y-d)$  where  $d = \min(y, 5-x)$

**Path cost:** Number of actions performed (1 per action), minimised to find the shortest sequence.

**State space:** **FINITE.** The 5L jug holds integer amounts  $\{0, 1, 2, 3, 4, 5\}$  (6 values) and the 3L jug holds  $\{0, 1, 2, 3\}$  (4 values), giving at most  $6 \times 4 = 24$  states. Not all are reachable, but the space is strictly bounded.

*One valid solution path (6 steps):*

$$(0, 0) \xrightarrow{\text{Fill 5L}} (5, 0) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (2, 3) \xrightarrow{\text{Empty 3L}} (2, 0) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (0, 2) \xrightarrow{\text{Fill 5L}} (5, 2) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (4, 2)$$

## Examination 2023 — Q2 (9 Marks)

### Problem Data

Table 1: State Transitions and Step

| Costs |      |      |
|-------|------|------|
| State | Next | Cost |
| A     | B    | 4    |
| A     | C    | 1    |
| B     | D    | 3    |
| B     | E    | 8    |
| C     | C    | 0    |
| C     | D    | 2    |
| C     | F    | 6    |
| D     | C    | 2    |
| D     | E    | 4    |
| E     | G    | 2    |
| F     | G    | 8    |

Table 2: Heuristic Estimates  $h(n)$ 

| Node | $h(n)$ |
|------|--------|
| A    | 8      |
| B    | 8      |
| C    | 6      |
| D    | 5      |
| E    | 1      |
| F    | 4      |
| G    | 0      |

**Important:** This is a **directed** graph — edges go one way only (State  $\rightarrow$  Next). Note the self-loop  $C \rightarrow C$  with cost 0 — this is a degenerate edge and should be ignored during search (it does not change the state or advance toward the goal). Also note the edge  $D \rightarrow C(2)$ : returning from D to C is allowed since the graph is directed — C is reachable from both A and D. No reverse edges from  $B \rightarrow A$ ,  $C \rightarrow A$ ,  $D \rightarrow B$ , etc. exist unless listed in the table above.

### Part a — Problem Formulations [2 marks]

**Question:** Give the initial state, goal test, operators, and path cost for each problem.

#### (i) Travelling Salesman Problem (TSP)

- State:** A pair  $(\ell, V)$  where  $\ell$  is the current city and  $V \subseteq \{\text{all } N \text{ cities}\}$  is the set of *unvisited* cities. *Example midway:* (Dhaka, {Chittagong, Sylhet}) means we are in Dhaka and still need to visit Chittagong and Sylhet.
- Initial state:** (start city, {all  $N-1$  other cities}).
- Goal test:**  $\ell = \text{start city}$  **and**  $V = \emptyset$  (returned to start after visiting all cities exactly once).
- Operators:** Travel( $\text{city}_i \rightarrow \text{city}_j$ ) where  $\text{city}_j \notin V$  or ( $V = \emptyset$  and  $\text{city}_j = \text{start}$ ). Each move removes  $\text{city}_j$  from  $V$ .
- Path cost:** Sum of distances (or costs) of all travelled roads. Goal: find the tour with **minimum total cost**.

**Complexity note:** With  $N$  cities there are  $(N-1)!/2$  distinct tours — factorial growth. TSP is NP-hard, so exhaustive search is infeasible for  $N > 20$ . Heuristic search ( $A^*$ ) with an admissible heuristic (e.g. minimum spanning tree of remaining cities) can find optimal or near-optimal tours for moderate  $N$ .

## (ii) Missionaries & Cannibals

**State:** A triple  $(M_L, C_L, B)$  where  $M_L$  = missionaries on left bank (0–3),  $C_L$  = cannibals on left bank (0–3),  $B \in \{L, R\}$  = boat position. The right bank has  $3-M_L$  missionaries and  $3-C_L$  cannibals.

**Initial state:**  $(3, 3, L)$  — all six people and the boat on the left bank.

**Goal test:**  $(0, 0, R)$  — all six people and the boat on the right bank.

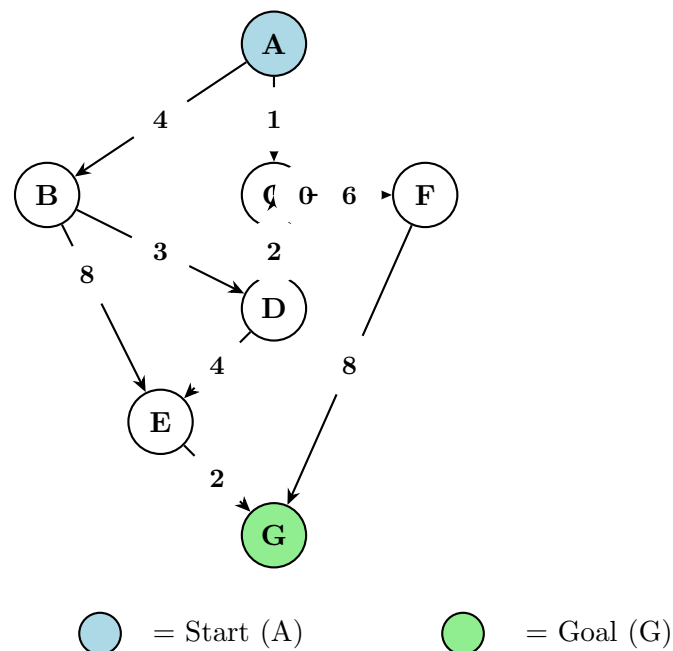
**Operators:** Move  $k$  missionaries and  $j$  cannibals across the river **by boat**, where  $1 \leq k+j \leq 2$  (boat capacity 2), and the resulting state must be **legal**: on both banks,  $M \geq C$  or  $M = 0$  (missionaries never outnumbered by cannibals on either side).

*Possible boat loads:*  $(1M, 0C)$ ,  $(2M, 0C)$ ,  $(0M, 1C)$ ,  $(0M, 2C)$ ,  $(1M, 1C)$ .

**Path cost:** Number of boat trips (1 per crossing). Goal: find a sequence with **minimum trips**.

**State space size:** Without constraints:  $4 \times 4 \times 2 = 32$  states. After legality filtering, only about 16–20 are valid. The problem has a known shortest solution of **11 crossings**.

### Part b(i) — State-Space Diagram [0.5 mark]



**Notes on the graph structure:**

- $C \rightarrow C(0)$  is a zero-cost self-loop — degenerate; ignored in search.
- $D \rightarrow C(2)$  creates a directed 2-cycle  $C \rightleftharpoons D$  ( $C \rightarrow D$  costs 2,  $D \rightarrow C$  costs 2).
- B has two outgoing edges:  $B \rightarrow D(3)$  and  $B \rightarrow E(8)$ .  $B \rightarrow E$  is an expensive direct path.
- The cheap route to the goal passes through  $D \rightarrow E(4) \rightarrow G(2)$ .
- From A, the best first step is C (cost 1), not B (cost 4).

**Part b(ii) — Search Algorithm Traces [4.5 marks]**

**Question:** Trace UCS, Greedy search, and A\* from A (initial) to G (goal). At each step: show the node being expanded and the fringe contents. Ties broken alphabetically.

**Algorithm 1 — Uniform-Cost Search (UCS)**

**Key idea:** Expand the node with the **lowest path cost**  $g(n)$ . Fringe is a min-priority queue sorted by  $g(n)$ . Once a node is expanded (closed), it is never re-expanded. If a cheaper path to a fringe node is found, update its cost.

**Fringe notation:** Node[ $g$ ] sorted ascending by  $g$ . **Closed set:** {nodes already expanded}.

| Step | Node | $g$ | Fringe after expansion (sorted by $g$ )                                                                                                             |
|------|------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | A    | 0   | C[1], B[4]<br>$A \rightarrow C$ : $g=1$ . $A \rightarrow B$ : $g=4$ . Both new.                                                                     |
| 2    | C    | 1   | D[3], B[4], F[7]<br>$C \rightarrow C$ : self-loop, ignore. $C \rightarrow D$ : $g=3$ new. $C \rightarrow F$ : $g=7$ new.                            |
| 3    | D    | 3   | B[4], E[7], F[7]<br>$D \rightarrow C$ : $g=5$ , C already closed $\Rightarrow$ skip. $D \rightarrow E$ : $g=7$ new.                                 |
| 4    | B    | 4   | E[7], F[7]<br>$B \rightarrow D$ : $g=7$ , D already closed $\Rightarrow$ skip. $B \rightarrow E$ : $g=12 > E[7]$ in fringe $\Rightarrow$ keep E[7]. |
| 5    | E    | 7   | F[7], G[9]<br>Tie E[7], F[7]: <b>E before F</b> (alphabetical). $E \rightarrow G$ : $g=9$ new.                                                      |
| 6    | F    | 7   | G[9]<br>$F \rightarrow G$ : $g=15 > G[9]$ in fringe $\Rightarrow$ keep G[9].                                                                        |
| 7    | G    | 9   | <b>GOAL FOUND</b>                                                                                                                                   |

**UCS Solution:**  $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$       **Total cost:**  $1 + 2 + 4 + 2 = 9$   
 Nodes expanded (in order): A, C, D, B, E, F, G    (7 nodes total)  
 UCS guarantees the optimal path because it explores in order of increasing  $g(n)$  and step costs are non-negative.

### Algorithm 2 — Greedy Best-First Search

**Key idea:** Expand the node with the **lowest heuristic**  $h(n)$ . Fringe sorted by  $h$  ascending. Path cost  $g(n)$  is *completely ignored*. This is fast but not guaranteed optimal.

**Fringe notation:** Node[ $h$ ] sorted ascending by  $h$ .

| Step | Node     | $h$ | Fringe after expansion (sorted by $h$ )                                                                |
|------|----------|-----|--------------------------------------------------------------------------------------------------------|
| 1    | <b>A</b> | 8   | C[6], B[8]<br>$A \rightarrow C$ : $h=6$ . $A \rightarrow B$ : $h=8$ .                                  |
| 2    | <b>C</b> | 6   | F[4], D[5], B[8]<br>$C \rightarrow F$ : $h=4$ . $C \rightarrow D$ : $h=5$ . $C \rightarrow C$ ignored. |
| 3    | <b>F</b> | 4   | G[0], D[5], B[8]<br>$F \rightarrow G$ : $h=0$ . Goal in fringe with lowest $h$ .                       |
| 4    | <b>G</b> | 0   | <b>GOAL FOUND</b>                                                                                      |

**Greedy Solution:**  $A \xrightarrow{1} C \xrightarrow{6} F \xrightarrow{8} G$  **Total cost:**  $1 + 6 + 8 = 15$  (suboptimal!)  
 Nodes expanded: A, C, F, G    (only 4 nodes — extremely fast)  
 Greedy blindly chased the heuristic: from C, F has  $h=4$  (lowest), so it went  $C \rightarrow F$ , but  $F \rightarrow G$  costs 8 — the most expensive path to the goal. It completely missed the cheap  $C \rightarrow D \rightarrow E \rightarrow G$  route (cost 9) because D's  $h=5$  looked slightly worse than F's  $h=4$ .

### Algorithm 3 — A\* Search

**Key idea:** Expand the node with the **lowest**  $f(n) = g(n) + h(n)$ . Fringe sorted by  $f$  ascending. Ties in  $f$  broken alphabetically. Closed set prevents re-expansion.

**Fringe notation:** Node[ $g, h, f$ ] sorted ascending by  $f$ .

| Step | Node | g | h | f | Fringe after expansion (sorted by f)                                                                                                                                                                                 |
|------|------|---|---|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | A    | 0 | 8 | 8 | C[1,6, <b>7</b> ], B[4,8, <b>12</b> ]<br>$A \rightarrow C$ : $f=1+6=7$ . $A \rightarrow B$ : $f=4+8=12$ .                                                                                                            |
| 2    | C    | 1 | 6 | 7 | D[3,5, <b>8</b> ], F[7,4, <b>11</b> ], B[4,8, <b>12</b> ]<br>$C \rightarrow D$ : $g=3, f=8$ . $C \rightarrow F$ : $g=7, f=11$ . $C \rightarrow C$ ignored.                                                           |
| 3    | D    | 3 | 5 | 8 | E[7,1, <b>8</b> ], F[7,4, <b>11</b> ], B[4,8, <b>12</b> ]<br>$D \rightarrow C$ : C closed $\Rightarrow$ skip. $D \rightarrow E$ : $g=7, f=8$ .<br>Tie D( $f=8$ ) and E( $f=8$ ): D already expanded, E is in fringe. |
| 4    | E    | 7 | 1 | 8 | G[9,0, <b>9</b> ], F[7,4, <b>11</b> ], B[4,8, <b>12</b> ]<br>$E \rightarrow G$ : $g=9, f=9$ .                                                                                                                        |
| 5    | G    | 9 | 0 | 9 | <b>GOAL FOUND</b>                                                                                                                                                                                                    |

**A\* Solution:**  $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$       **Total cost:**  $1 + 2 + 4 + 2 = 9$  (optimal!)

Nodes expanded: A, C, D, E, G (only 5 nodes — fewer than UCS's 7)

A\* found the optimal path and expanded **only 5 nodes** compared to UCS's 7, because the heuristic  $h(n)$  guided it away from the expensive B and F branches. Note that **B was never expanded** — its  $f=12$  was always worse than the alternatives ahead of it in the fringe.

### Algorithm Comparison for 2023 Graph:

| Algorithm | Path Found                                                  | Cost     | Nodes Expanded | Optimal?   |
|-----------|-------------------------------------------------------------|----------|----------------|------------|
| UCS       | $A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$ | <b>9</b> | 7              | <b>Yes</b> |
| Greedy    | $A \rightarrow C \rightarrow F \rightarrow G$               | 15       | 4              | No         |
| A*        | $A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$ | <b>9</b> | 5              | <b>Yes</b> |

**Why does A\* outperform UCS here?** The heuristic  $h(n)$  provides a useful lower bound. After expanding A and C, A\* sees that D has  $f=8$  (promising —  $h(D)=5$  is close to the true remaining cost of 6) while B has  $f=12$  (unpromising —  $h(B)=8$  overestimates its potential). A\* expands D next and finds the optimal path without ever touching B. UCS, lacking a heuristic, expands B unnecessarily at step 4 before reaching the goal.

**Admissibility check:** Is  $h(n)$  admissible? Let's verify:

- $h(A)=8 \leq h^*(A)=9$  ✓
- $h(C)=6 \leq h^*(C)=8$  ✓
- $h(D)=5 \leq h^*(D)=6$  ✓
- $h(E)=1 \leq h^*(E)=2$  ✓
- $h(B)=8 \leq h^*(B)=5$ ? **No!**  $h^*(B) = \min(3+4+2, 8+2) = 9$ . Wait —  $B \rightarrow D \rightarrow E \rightarrow G = 3+4+2 = 9$ , so  $h(B)=8 < 9$  ✓. And  $B \rightarrow E \rightarrow G = 8+2 = 10$ . So  $h^*(B)=9$ ,  $h(B)=8$  is admissible.

- $h(F)=4 \leq h^*(F)=8$  ✓

**Yes, the heuristic is admissible** —  $h(n) \leq h^*(n)$  for all nodes. This is why A\* found the optimal solution.

### Part c — IDDFS: Synthesising Benefits of BFS and DFS [2 marks]

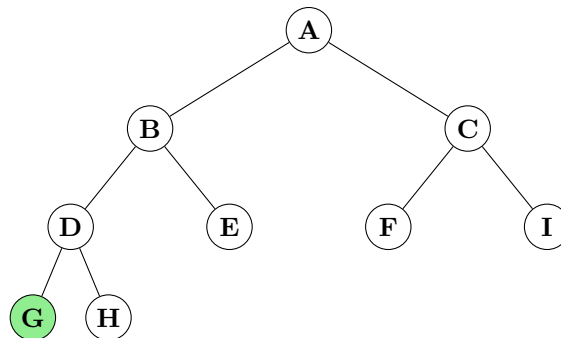
**Question:** Illustrate with an appropriate example how IDDFS synthesises the benefits of both BFS and DFS — achieving optimality, completeness, and linear space complexity. Nevertheless, time complexity deteriorates in comparison to both BFS and DFS.

#### The Core Idea

IDDFS (Iterative Deepening Depth-First Search) runs DFS repeatedly with an increasing depth cutoff: first  $L=0$ , then  $L=1$ ,  $L=2$ ,  $\dots$ , until the goal is found. At each iteration, the search is a standard depth-limited DFS from the root.

#### Illustrative Example

Consider the following binary tree where G (shaded) is the shallowest goal at depth 3:



#### IDDFS Execution:

| L | Nodes visited (DFS order)      | Outcome                                       |
|---|--------------------------------|-----------------------------------------------|
| 0 | A                              | $A \neq G$ . No solution at depth 0.          |
| 1 | A, B, C                        | Neither B nor C is G. No solution at depth 1. |
| 2 | A, B, D, E, C, F, I            | No goal at depth 2.                           |
| 3 | A, B, D, G ← <b>GOAL FOUND</b> | <b>Found!</b> Shallowest goal at depth 3.     |

## How IDDFS Inherits Each Algorithm's Strengths

| Property            | BFS                                                                              | DFS                                                     | IDDFS                                                                                                                                                            |
|---------------------|----------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Completeness</b> | ✓ Complete (finds solution if one exists)                                        | × Incomplete (may loop forever on infinite branches)    | ✓ <b>Complete</b> — like Depth-limited DFS and avoids infinite loops because it backtracks at the cutoff                                                         |
| <b>Optimality</b>   | ✓ Optimal for uniform step costs (finds shallowest solution)                     | × Not optimal (may find a deep solution first)          | ✓ <b>Optimal for uniform costs</b> — like BFS. Because it explores all nodes at depth $d-1$ before any at depth $d$ , the first solution found is the shallowest |
| <b>Space</b>        | × $O(b^d)$ — stores entire frontier. For $b=10, d=10$ , that's 10 billion nodes. | ✓ $O(bm)$ — stores only one path + siblings.            | ✓ $O(bd)$ — like DFS. Only the current path (depth $d$ ) plus $b-1$ siblings at each level are in memory. $b=10, d=10$ : $\approx 100$ nodes                     |
| <b>Time</b>         | $O(b^d)$ — visits each node once                                                 | $O(b^m)$ — may go very deep, but each node visited once | × $O(b^d)$ — <b>worse constant factor</b> . Nodes near the root are revisited many times. Overhead factor is large (e.g. for $b=10$ , about 10 times BFS).       |

## Why Time Complexity Deteriorates

The key weakness of IDDFS is **repeated work**. Consider the root node A: it is expanded in *every* iteration ( $L=0, 1, 2, 3$ ) — 4 times. Node B is expanded at  $L=2$  and  $L=3$  — twice. Only the deepest nodes (depth  $L$ ) are expanded just once.

However, in a tree with branching factor  $b$ , the **number of nodes at the deepest level dominates**:

$$\begin{aligned}
 \text{Total expansions} &= (d+1) \cdot 1 + d \cdot b + (d-1) \cdot b^2 + \dots + 1 \cdot b^d \\
 &= b^d \left( 1 + \frac{1}{b} + \frac{1}{b^2} + \dots \right) \approx b^d \cdot \frac{b}{b-1}
 \end{aligned}$$

For  $b = 10$ , the overhead is only  $\frac{10}{9} \approx 1.11\times$  — a small price for the enormous space savings ( $O(bd)$  vs  $O(b^d)$ ). This is why IDDFS is the **preferred uninformed search strategy** when the state space is large and step costs are uniform.



## When IDDFS Is NOT Optimal

IDDFS finds the **shallowest** goal (fewest edges), not the lowest-cost goal. With non-uniform step costs, a deeper path with lower total cost may exist. In such cases, UCS or A\* is required. For the 2024 exam graph, IDDFS found  $A \rightarrow B \rightarrow E \rightarrow G$  (cost 16, 3 hops) while UCS found  $A \rightarrow C \rightarrow H \rightarrow G$  (cost 13, 3 hops) — both at the same depth, but UCS found the cheaper one because it considers edge costs, not just depth.

### Exam summary — IDDFS:

- **Completeness:** Inherited from BFS — guaranteed to find a solution if one exists.
- **Optimality:** Inherited from BFS — optimal for *uniform* step costs (finds shallowest solution).
- **Space:** Inherited from DFS —  $O(bd)$  linear space, practical for large search spaces.
- **Time:** Worse than both BFS and DFS individually — nodes near the root are re-expanded each iteration. However, the overhead is only a constant factor ( $\approx \frac{b}{b-1}$ ), acceptable given the space savings.

## Examination 2022

---

**Note:** Search questions in 2022 are **scattered across two sections**:

- **Section A, Q.A-4(a):** DFS vs BFS theory (2 marks)
- **Section B, Q.B-4(b):** Graph trace — UCS, Greedy, A\* (3 marks) — *identical search space to 2023*

### Section A — Q.A-4(a): DFS vs BFS [2 marks]

**Question:** What is an AI Technique? Write down the advantages and disadvantages of Depth-First Search and Breadth-First Search.

### AI Technique

An **AI Technique** is a method that exploits knowledge, represented in a form that:

1. **Captures generalisations** — groups situations sharing important properties together, avoiding separate representations for every individual case.
2. **Can be understood** by the people providing it — knowledge must be inspectable and modifiable.
3. **Can be easily modified** to correct errors and reflect new information.
4. **Can be used in many situations** even if not perfectly complete or accurate.
5. **Reduces the search space** by narrowing the range of possibilities that must be considered.

In short: an AI technique is a **knowledge-driven method for controlling search** so that solutions are found efficiently without exhaustive enumeration of all possibilities. DFS and BFS are two fundamental *uninformed* (blind) search techniques — they use no domain-specific knowledge, only the structure of the search tree.

**Depth-First Search (DFS)**

| Advantages                                                                                                                                          | Disadvantages                                                                                                                         |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Low memory:</b> Stores only the current path ( $b$ siblings per level), giving $O(bm)$ space. For deep trees, this is the decisive advantage. | <b>1. Not complete:</b> Fails on infinite-depth trees. DFS may follow an infinite branch forever and never find a solution elsewhere. |
| <b>2. Simple implementation:</b> Easy to code recursively or with a stack (LIFO).                                                                   | <b>2. Not optimal:</b> The first solution found may be very deep in the tree. DFS has no mechanism to prefer shorter paths.           |
| <b>3. Fast to a solution if lucky:</b> If the goal is deep and branching is low, DFS may find it with very few node expansions.                     | <b>3. Susceptible to loops:</b> Without a closed/visited set, DFS can cycle infinitely.                                               |
| <b>4. Backtracking-friendly:</b> Natural fit for constraint satisfaction and puzzle-solving where partial solutions are built and abandoned.        | <b>4. May miss shallow goals:</b> DFS goes deep first — a goal at depth 2 may be found only after exhausting paths to depth 50.       |

**Breadth-First Search (BFS)**

| Advantages                                                                                                              | Disadvantages                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Complete:</b> Guaranteed to find a solution if one exists (provided $b$ is finite).                               | <b>1. Exponential memory:</b> Must store the entire frontier — $O(b^d)$ space. For $b=10$ , $d=12$ : 1 trillion nodes in memory. This makes BFS <b>infeasible</b> for most real problems.  |
| <b>2. Optimal (uniform cost):</b> Finds the shallowest solution. If step costs are all equal, this is the optimal cost. | <b>2. Exponential time:</b> $O(b^d)$ — same as the space requirement. Practical only for shallow trees.                                                                                    |
| <b>3. Systematic:</b> Processes the entire tree level by level — no path is ever skipped or overlooked.                 | <b>3. Slow to find deep goals:</b> Must explore all nodes at depth $d-1$ before checking any at depth $d$ . If the goal is at depth 10, all 9 shallow levels must be fully expanded first. |
| <b>4. Good for shallow goals:</b> If the goal is near the root, BFS finds it quickly.                                   | <b>4. Queue implementation overhead:</b> Maintaining the frontier as a FIFO queue with $O(1)$ enqueue/dequeue while also tracking visited states adds complexity.                          |

**When to Use Which**

| Criterion                   | Use DFS                     | Use BFS                                |
|-----------------------------|-----------------------------|----------------------------------------|
| Memory is limited           | <b>Yes</b> — $O(bm)$ space  | No — $O(b^d)$ space                    |
| Goal is known to be deep    | <b>Yes</b> — goes deep fast | No — explores all shallow levels first |
| Goal is known to be shallow | No — may waste time deep    | <b>Yes</b> — finds it early            |
| Need shortest path          | No — not optimal            | <b>Yes</b> (uniform costs)             |
| Infinite / very deep tree   | No — may not terminate      | <b>Yes</b> — complete                  |

**The IDDFS resolution:** Iterative Deepening DFS combines the space advantage of DFS ( $O(bd)$  memory) with the completeness and optimality of BFS. For most practical uninformed search problems where memory is the bottleneck, **IDDFS is the preferred choice**. It was tested in 2024 Q2, 2023 Q2(c), 2021 Q3(b), and 2020

Q3(b) — a core exam pattern.

### Section B — Q.B-4(b): Graph Trace [3 marks]

**Question:** Suppose you have the following search space (State, Next, Cost as given in Table 1,  $h(n)$  as given in Table 2). Assume initial state A and goal state G. Show how each of the following search strategies would create a search tree to find a path from A to G: **Uniform cost, Greedy search, and A\***. At each step show which node is being expanded and the content of fringe.

The **2022 search space is identical to the 2023 search space** (same states, same edges, same  $h(n)$  values). The full step-by-step traces for UCS, Greedy, and A\* on this graph are given in **Section 2.3** (Examination 2023, Part b(ii)). Below is the summary.

| Algorithm | Path              | Cost     | Nodes Expanded          |
|-----------|-------------------|----------|-------------------------|
| UCS       | A → C → D → E → G | <b>9</b> | 7 (A, C, D, B, E, F, G) |
| Greedy    | A → C → F → G     | 15       | 4 (A, C, F, G)          |
| A*        | A → C → D → E → G | <b>9</b> | 5 (A, C, D, E, G)       |

**Key observation — same graph appears 3 times:** The identical search space (A→B(4), A→C(1), B→D(3), B→E(8), C→D(2), C→F(6), D→C(2), D→E(4), E→G(2), F→G(8)) with the same  $h(n)$  values is tested in **2020 Q2, 2022 B-4(b), and 2023 Q2**. Mastering the traces on this one graph gives you answers for 3 separate exam years. The cheapest path is always A→C→D→E→G costing 9.